TruthPad User Manual

> A lightweight, fast, plugin-based development editor — C++20 + pure Win32 + Scintilla,
>
> a 9.1 MB single exe, no Node.js in the core or the build chain. This manual covers the current
>
> Beta channel build (Help → About TruthPad shows 0.2.6).
>
> **How to read**: `View → Markdown Live Preview` means a menu path — the menu opens from the
>
> hamburger button ☰ at the top left; `Ctrl+F` means a keyboard shortcut;
>
> `> 🖼 Figure x-y` marks a **screenshot to be added later** (the full list is in Appendix D);
>
> the text is complete without the images.

---

1. Highlights

1.1 Open from the context menu = minimal mode, milliseconds

The file association the installer registers is `TruthPad.exe --quick "%1"`. Right-clicking (or double-clicking) a text file and choosing TruthPad:

- **does not restore the previous workspace** (no sidebar, no terminal, no AI panel);
- **does not add an empty "Untitled" tab** — the point is to see and edit that one file fast;
- affects only this session: **`config.ini` is left alone**, so the next normal launch still restores your workspace.

Measured on the project's own acceptance runs: hot start to window **< 200 ms**, cold start including file load **< 700 ms**, idle memory around 16 MB.

Double-clicking the exe (or launching with no file) still gives the full UI and opens a self-drawn **Welcome** tab. To get minimal mode by hand, add `--quick` yourself.

If you associated TruthPad through Windows' "Open with → Choose another app" (those entries are just `"%1"`, with no `--quick`), opening a file degrades to the full UI — run **Help → Repair Open-with Associations** to rewrite it as `"<exe>" --quick "%1"` (current user only; no admin needed).

> 🖼 **Figure 1-1** — Minimal mode: a single open `.md` tab, no sidebar, no welcome page, no untitled tab

1.2 Large Markdown files + built-in PDF / HTML export

- **Large files never block typing**: from 4 MB up, the editor turns syntax highlighting off to

save memory (the status bar says "Large file: syntax highlighting off (memory saving)"). The preview renders in a separate WebView2 process with a 250 ms debounce, and the Mermaid engine is loaded only when the document actually contains a mermaid block.

- **Built-in export**: right-click in the editor, the preview, or the right-hand Preview pane and choose **Export PDF** or **Export HTML**. The output lands **next to the source .md, with the same name** (`report.md → report.pdf` / `report.html`, overwriting silently if it exists). PDF is A4 portrait, 17 mm top/bottom margins, 16 mm left/right, backgrounds printed, no headers/footers, forced light theme. HTML is a single file (styles inlined, Mermaid already converted to SVG, image paths made absolute) — ready to hand to someone else.

> 🖼 **Figure 1-2** — Markdown with math and a Mermaid diagram: source on the left, live preview on the right, with the export menu open

1.3 Everything extra is loaded on demand

- **Startup loads no plugin DLLs at all**: it reads plain-text manifests (or the manifest embedded in a DLL, opened in "data mode" — no code executed), pre-builds the menus, and loads a plugin only when you click it.
- **Panels and web views are created when first needed**: plugin sidebar views, bottom-dock plugin tabs, the right-hand AI panel (loaded only after you have actually used it), and every WebView2 (Settings / plugin pages / Markdown preview / right Preview pane / browser).
- Measured: with the right panel collapsed, startup spins up **zero `msedgewebview2` child processes**; there is a dedicated regression script watching this.

> 🖼 **Figure 1-3** — Plugin manager: each plugin declares its capabilities and how it loads (lazy / disabled / uninstalled)

1.4 PDF reader + AI quick read / deep read

- **Annotations**: underline, strikethrough, text highlight, area highlight, freehand highlight, line / rectangle / ellipse / cloud / polygon / freehand pen, text notes / text boxes / callouts / sticky notes — with a 50-step undo stack, and they **save back into the PDF file**.
- **Paper quick read / Paper deep read** (the two rightmost buttons on the status bar, using the shared AI model):

	Paper quick read	Paper deep read

Goal	Decide **whether the paper deserves your time**	Make sure you **actually understand it**
Length	Restrained (1,500–3,000 characters, not an abstract)	No ceiling, no padding allowed
Content	Skeleton of key points, jargon in plain words, intuition behind the core formulas	Real background, prior work named with references, every symbol explained, Mermaid diagrams where needed

- Products land in `Documents\TruthPad\PDF\<pdf-name>-<identity-hash8>-speedread.md` (deep read: `-deepread.md`) and open in the right-hand **Preview** pane. **Clicking again on the same document hits the cache — 0 model calls** — identity is the PDF's embedded document ID, so **annotating or bookmarking the PDF does not make it a "new document"**.

> 🖼 **Figure 1-4** — PDF reader with its thumbnail sidebar, toolbar and annotations, plus a speed-read draft generating in the right Preview pane

1.5 Browser plugin: subtitles for videos → Chinese translation → lecture notes

- Open a video page in the built-in browser (bilibili is the fully verified pipeline) and click **Subtitles** in the toolbar: **local whisper transcribes it segment by segment** (models download in-app; **audio never leaves your machine**), and a subtitle overlay shows the original text.
- When the whole episode is transcribed it goes to DeepSeek in one call: foreign-language audio is **translated into Chinese subtitles**; Chinese audio is only **proofread** (recognition slips and formula notation corrected).
- Right-click → **Generate article**: the transcript becomes a **long lecture note** (formulas filled in, Mermaid diagrams, a closing summary / checklist / discussion questions). **The writing style is detected from the content** (Academic lecture / Light essay / News report), and you can also pin it by hand.
- An article that already exists opens instantly (0 model calls); an unfinished one **resumes where it stopped**.

> 🖼 **Figure 1-5** — A bilibili lecture playing with the Chinese subtitle overlay, and the note being generated in the right preview

---

2. Text Editor (the host application)

2.1 Install and launch

- The **installer** is a self-drawn one-button dark window. It installs to

  `%LOCALAPPDATA%\Programs\TruthPad` — no admin rights, no restart — creates a Start-menu shortcut, an optional desktop shortcut, the `*.truthpad.json` file association, and registers an uninstall entry under Settings → Apps. Uninstalling removes program files, shortcuts and the association; settings and recent lists in `%APPDATA%\TruthPad` are kept.

- **Command line**: `TruthPad.exe [--quick] <file or project …>`. Other arguments starting with `-`

  or `/` are ignored; an argument ending in `.truthpad.json` opens as a project file, anything else as a normal file.

- **Launch position**: centered on the work area of the monitor **under the cursor** — on a

  multi-monitor setup the window appears where you started it, and it is clamped so the taskbar cannot cover it.

- **Welcome page**: opened on a normal launch (no file given) — four "Get Started" cards

  (New File / Open File / Open Folder / Open Project), recent files, and key shortcuts. Closing it, or closing the **last** tab, does **not** create a blank document; when no tab is left the editor area shows an empty-state hint (artwork + shortcuts), and **double-clicking the empty area creates a new file**.

> 🖼️ **Figure 2-1** — Welcome page: Get Started cards, Recent, and Shortcuts

2.2 UI overview

- **Self-drawn title bar**: the hamburger menu ☰ and "TruthPad — project name" on the left; three

  layout buttons on the right (left sidebar / bottom terminal / right AI panel — filled = shown, hollow = hidden) plus minimize / maximize / close. Drag the empty area to move the window, double-click to maximize.

- **Left sidebar**: a permanent 44 px icon rail. The built-ins come first (**Project** file tree and

  Search), plugin entries follow by priority; clicking an icon expands the sidebar on that panel; collapse it with the title-bar button or `Ctrl+B`.

- **Editor area**: tab bar plus the editor (line numbers, folding, lexer chosen by file extension).
- **Bottom panel**: plugin tabs (such as **Terminal**) plus an **Output** tab (Ctrl+backtick to show/hide).
- **Right panel**: two tabs — **Preview** (everything plugins generate opens here) and **AI**.
- **Status bar**: on the left, the message, line/column (with selection length when there is one),

  language, encoding and line endings; on the right, plugin buttons (e.g. the PDF quick/deep read) and, at the far right, the double-arrow button whose tooltip reads "Collapse the status bar into the

sidebar (click)".

> 🖼 **Figure 2-2** — UI overview (title bar, icon rail, tab bar, editor, status bar, right panel)

2.3 Tabs and split panes

- **Tabs**: a dirty dot for unsaved changes; middle-click closes; double-clicking empty tab-bar space creates a new file; the wheel scrolls the tab bar horizontally.
- **Tab context menu**: Close / Close Others / Close Tabs to the Right / Split Right / Split Down.
- **Up to 4 editor panes** (any mix of rows and columns):
- drag a tab to within **25% of a pane edge** to split that pane in that direction; drop it in the **middle 50%** to move the tab into that pane (a drop hint shows where it will land);
- when a pane's **last tab** is dragged away or closed, the pane dissolves and its sibling fills the space;
- every pane has its own tab bar and its own draggable splitter; **focus follows the pane you click into** — the find bar, the status bar and Save all act on the focused pane.

> 🖼 **Figure 2-3** — Dragging a tab to split: drop hint and a finished four-pane layout

2.4 Files, encodings and line endings

- **Encoding detection**: UTF-8 (with or without BOM), UTF-16 LE / BE (BOM), any valid UTF-8, and the system ANSI code page (GBK/CP936 on Simplified-Chinese Windows); if ANSI fails it retries in a lenient GB18030 mode (the status bar then notes "non-standard file encoding; read in lenient GBK mode").
- **Saving writes the original encoding back** — GBK sources in Keil or GCC projects stay GBK. If the encoding cannot represent a character, you are asked whether to save as UTF-8 instead.
- **Line endings** (CRLF / LF / CR) are preserved; the status bar displays the current one read-only.
- New documents are named "Untitled", then "Untitled-2" and so on; closing a document with unsaved changes asks first.

2.5 Find / Replace / Search in files

Editor find bar (`Ctrl+F` for find, `Ctrl+H` with the replace row):

- opens pre-filled with the selected single-line text; `Esc` closes it and returns focus to the editor.
- Buttons: `▲` previous, `▼` next, `Aa` match case, `W` whole word, `.*` regex, ⇄ show/hide the replace row, ✕ close.
- Keyboard: Enter = find next, Shift+Enter = find previous, `F3` / `Shift+F3` the same; Enter in the replace box performs Replace. All three options start off.

Search in Files (left sidebar, `Ctrl+Shift+F`, or View → Search in Files):

- search-as-you-type (450 ms debounce); toggles for `Aa`, Whole Word and `.*` (regex); **Replace** expands the replace row, **Filter** expands "files to include / exclude" glob boxes.
- Results are **grouped by file** (collapsible, with hit counts); clicking a hit **jumps to that line** (whole line selected).
- **Replace All** asks for confirmation first (**it cannot be undone**), skips files with unsaved changes and files that cannot be written back in their original encoding, and refreshes open, clean tabs afterwards.
- The search skips `.git`, `node_modules`, `build`, `dist` and friends, binary files (a NUL in the first 8 KB) and files over 4 MB; UTF-16 files are reported as skipped. A single file is capped at 200 hits, the whole search at 20,000 (truncation is reported). Only **local** workspace roots are searched for now (remote `wsl://` roots are not included).

> **Figure 2-4** — Search results in the sidebar: grouped by file, replace row expanded

2.6 Workspace tree (Project)

The panel is titled **Project**; you can also drag a folder from Explorer into the window to add it as a workspace root.

- Header buttons: ⊕ open folder, ⟳ refresh (`F5`, focus must be in the tree).
- Single-click a file = preview open (not pinned); double-click = pinned open; double-click a folder expands/collapses it.
- **Multi-select**: `Ctrl+click` to add/remove, `Shift+click` for a range, `Ctrl+A` for all, `Esc` to clear.
- **Drag to move**: drop files/folders onto a folder row to move them there (a translucent card follows the cursor and the drop target highlights; same-name collisions and dropping a folder into itself are rejected).
- **Context menu** (fixed order):

`Open / Show in File Explorer | Copy / Paste | Copy Relative Path / Copy Absolute Path | New File / New Folder | Rename / Delete`

A workspace root ends with **Remove from Workspace** (it does **not** delete anything from disk); a single selected folder additionally gets Expand/Collapse.

- **New**: creates a temporary name first ("New File.txt" / "New Folder", auto-numbered `(2)` when taken) and renames in place; `Esc` keeps the temporary name.
- **Rename**: `F2`; Enter commits, `Esc` cancels, and clicking elsewhere also accepts the name.

  Illegal names (empty, containing `\ / : * ? " < > |`, duplicates, reserved Windows device names, trailing dot or space) are rejected and the original name is kept.
- **Delete**: `Delete` — straight to the **Recycle Bin** (no confirmation; folders recursively;

  any open tabs are closed).
- **Copy / Paste**: `Ctrl+C` / `Ctrl+X` / `Ctrl+V`, interchangeable with File Explorer; duplicates

  become `xxx (2).ext`.
- Write operations on remote (`wsl://`) nodes are greyed out.

> 🖼️ **Figure 2-5** — Workspace tree context menu and the drag card

2.7 Terminal and Output

- **Terminal plugin**: the top-level **Terminal** menu opens cmd / PowerShell / SSH / TTY; all

  sessions live in the bottom **Terminal** tabs, not in editor tabs. Ctrl+backtick toggles the bottom panel.
- **SSH**: enter host, port and user. A password may be supplied once at connect time, or left empty;

  the terminal then shows `user@host's password:` and reads it without echo. SSH sessions support disconnect and terminal resizing.
- **Serial TTY**: pick a port and baud rate (automatic detection is supported), plus data bits,

  parity, stop bits, flow control and UTF-8 output.
- **SFTP**: after an SSH connection, open the SFTP sidebar from the left rail. Type a remote path in

  the path box; file/folder icons match the netdisk plugin. Clicking the sidebar header ✕ disconnects SFTP and removes the rail icon; hiding the sidebar from the title bar does not disconnect. `sftp://` is internal to TruthPad and does not mount a drive.
- **Output panel**: multiple channels (TruthPad, Build, and plugin channels such as **PDF** and

  Browser). Each line reads `timestamp [module] [level]: text`; the **level** button at the far right of the tab strip selects the **minimum level shown** (Trace / Debug / Info / Warning / Error / All,

default Info). A plugin's channel tab appears automatically the first time it writes output. Right-click the panel to clear or copy everything.

> **Figure 2-6** — Terminal panel (cmd / PowerShell / SSH / TTY) and the SFTP file sidebar

2.8 Project files `*.truthpad.json`

A project file can bring up several roots plus a set of settings at once (File → Open Project File…, `Ctrl+Alt+O`; opened projects join **Recent Projects**):

```
{
  "name": "My firmware project",
  "folders": [
    "firmware",
    { "path": "../lib", "name": "Shared libraries" }
  ],
  "settings": { "theme": "dark", "defaultEncoding": "utf8", "tabWidth": 4 },
  "wsl": { "distro": "Ubuntu", "remotePath": "/home/me/proj" }
}
```

- `name` shows up in the title bar as "TruthPad — name" (defaults to the project file's folder name);
- paths in `folders` are relative to the project file's folder and may carry a display name;
- `settings` recognises `theme` (dark/light), `defaultEncoding` (utf8/utf8bom/utf16le/ansi) and `tabWidth` (1–16, default 4);
- the `wsl` block is used by the WSL plugin to mount a remote folder.

2.9 Settings (`Ctrl+,`)

Six sections on the left: **Appearance / Editor / Plugins / Models / Default apps / About**.

- **Appearance**: Language (applies immediately), File log level (Trace default / Debug / Info / Warning / Error — lines below it are not written to the log files, plugin logs included), Theme (Dark / Light, applies immediately), Color file (buttons **Reload colors** and **Edit theme.json**).
- **Editor**: Font size (currently 14px), Tab width (4 spaces; a project file's `settings.tabWidth` can override it).
- **Plugins**: explanation of the plugins folder, a **Manage plugins…** button, and the "N discovered · M loaded" counter.
- **Models**: see 5.5 — one shared model configuration for every AI plugin.
- **Default apps**: choose the file types associated with TruthPad (PDF / source / images / video).
- **About**: version and components (Scintilla 5.6.7 · Lexilla 5.5.4 · WebView2 · libvterm 0.3.3 · libssh2 1.11.1).

> **Figure 2-7** — Settings: the Appearance and Models sections

2.10 Theme and colors

- The color file is `%APPDATA%\TruthPad\theme.json`, shaped as `{"dark":{…},"light":{…}}` and

  overriding the built-in defaults key by key (on first run a file with `_version` is written out for you to edit).
- Built-in defaults: syntax colors follow VS Code Dark+/Light+; the dark background deliberately avoids `#1E1E1E` and uses greys such as `#111214`.
- No restart needed: switch in Settings, or use **View** → **Reload Colors**; plugin UIs follow the theme too.
- A broken file never crashes anything: malformed JSON falls back to the defaults, and the problem is recorded in `theme.log`.

2.11 Language

- Two UI languages ship today: **简体中文** and **English (US)**.
- Switch under Settings → Appearance → Language (menus, status bar and plugin pages all follow, immediately).
- By default the app follows the system language; if a language file is missing it falls back to English.

2.12 Plugin manager and plugin source

- Open it from the menu (**Plugins…** `Ctrl+Shift+P`) or Settings → **Manage plugins…**.
- The page has **Installed** and **Available** categories; at the top you can fill in the

  plugin source URL (a static folder holding `index.json` and the plugin archives) and click **Refresh list**, plus **Install from File…**, **Rescan Folder**, **Open Plugins Folder**, and a search box (name / author / description).
- Plugin states: Loaded / Lazy / Disabled / Manifest error / Not installed /

  Legacy plugin · loads at startup; capability tags: Files / Remote / Events / Commands / Panel; kinds: Viewer / Tool / Provider / Agent.
- Buttons: **Load Now / Enable / Disable / Uninstall / Install / Reinstall / Configure**.

  Disabling also removes its menu commands and slots; uninstalling goes to the **Recycle Bin** (recoverable), and a plugin with live child processes is refused.

- The **Configure** dialog is generated from the plugin's declared settings and offers

  Restore defaults / Cancel / Save; changes apply immediately for loaded plugins and on next use for unloaded ones.

- Installing from the plugin source verifies size and sha256, extracts, validates the manifest, and

  then installs.

> 🖼️ **Figure 2-8** — Plugin manager: available plugins and the plugin settings dialog

2.13 Host keyboard shortcuts

Shortcut	Action
`Ctrl+N` / `Ctrl+O`	New file / Open file
`Ctrl+Shift+O` / `Ctrl+Alt+O`	Open folder as workspace / Open project file
`Ctrl+S` / `Ctrl+Shift+S`	Save / Save As
`Ctrl+W`	Close current tab
`Ctrl+Z` / `Ctrl+Y`	Undo / Redo
`Ctrl+X` / `Ctrl+C` / `Ctrl+V` / `Ctrl+A`	Cut / Copy / Paste / Select All
`Ctrl+F` / `Ctrl+H`	Find / Replace
`F3` / `Shift+F3`	Find next / previous (also inside the find bar)
`Ctrl+Shift+F`	Search in files (left sidebar)
`Ctrl+B` / `Ctrl+Alt+B`	Sidebar / AI assistant panel
`Ctrl+Shift+M`	Markdown live preview (editor ↔ split)
Ctrl+backtick	Show/hide the terminal panel
`Ctrl+Shift+T`	New terminal session
`Ctrl+Shift+P` / `Ctrl+,`	Plugin manager / Settings
`Ctrl+Tab` / `Ctrl+Shift+Tab`	Next / previous tab (cycles within the focused pane)
`F5`	Refresh the workspace tree (when focus is in the tree)

The View menu also lists "Toggle Fold `Ctrl+Shift+[` ", "Fold All" and "Expand All".

---

3. Markdown Editor

3.1 Opening a file and the three modes

- Opening a `.md` file starts in **Editor only** mode; documents generated by plugins (lecture notes, quick/deep reads) start in **Preview only**. The three modes are mutually exclusive:

Mode	What you get
Editor only	Pure writing; lightest (no WebView2 is created)
Split	Write on the left, watch on the right
Preview only	Distraction-free reading

- Where to switch: the editor context menu / the preview context menu — the items read `Preview only`, `Editor only` and `Split (editor | preview)`, and the current mode is checked; the host menu **View** → **Markdown Live Preview** `Ctrl+Shift+M` toggles **editor** ↔ **split**.
- The preview top bar shows the title "Markdown Preview", the two scroll-sync arrows, and a `×` on the right that closes the preview and returns to editor-only. Non-Markdown files do not get these menu items (you get "The current file is not Markdown; no preview").

> 🖼 **Figure 3-1** — Split mode: source left, preview right, with the two sync arrows on the preview top bar

3.2 What the preview renders

The preview is a WebView2 page, so it renders like a modern Chromium page:

- **GFM-flavoured Markdown** (markdown-it 14.3.2): tables, task lists, autolinks. A single newline does **not** break a line (leave a blank line, or end the line with two spaces).
- **Heading anchors** and `[TOC]`; the ☰ button at the top right of the preview opens a **contents sidebar** (h1–h4; click to jump smoothly, the current section highlights as you scroll; the button is hidden when the document has no usable headings).
- **Code highlighting** (highlight.js 11.12.0) and **Mermaid diagrams** (3.4.12) — the Mermaid engine is only loaded when the document actually contains a mermaid block.
- **Math** (KaTeX 0.16.11): `$…$`, `$$…$$`, `\(…\)` and `\[…\]` **are all supported**; a malformed formula shows an error instead of breaking the page.
- **Relative images and attachments** resolve against the document's own folder.

> 🖼 **Figure 3-2** — Preview: math, a Mermaid diagram, highlighted code, contents sidebar

3.3 Scroll sync and cursor marking

- Two arrows on the preview top bar: `→` **= the editor scrolls the preview**,

  `←` **= the preview scrolls the editor**. Bright (main text color) = on, dimmed = off; **both start on**. Turning one on aligns the two sides immediately.

- Sync is anchored on **content lines**, not on an overall ratio, so long documents never drift.
- The editor caret is drawn as a vertical bar in the preview, and the selection is highlighted there

  — each side can find the other.

3.4 Copy / Select All inside the preview

The preview context menu offers **Copy** (greyed out when nothing is selected) and **Select All**. Copy writes the selected body text to the system clipboard and reports the character count in the status bar.

3.5 Exporting PDF / HTML

- **Where**: three context menus with identical items — the editor, the preview, and the right-hand

  Preview pane — each offering **Export PDF** and **Export HTML**.

- **Output**: next to the source `.md`, **same name** (`report.md → report.pdf` / `report.html`),

  and an existing file is **overwritten** (both the status bar and the Output panel say so).

- **How the PDF is made**: the already-laid-out preview page is sent to WebView2's own print

  pipeline — math is already typeset and Mermaid is already SVG, so it is WYSIWYG. Page setup is fixed: **A4 portrait, 17 mm top/bottom and 16 mm left/right margins, 100% scale, backgrounds printed (code-block tints survive), no headers or footers, forced light theme**; body text uses Arial with YaHei fallback, code uses Consolas.

- **HTML is a single file**: styles inlined, image paths made absolute (`file:///…`), Mermaid

  inlined as SVG, forced light theme. It carries no page scripts, so opening it does not get you the interactive contents sidebar.

- **Confirmation**: the status bar plus "PDF generated: …" in the Output panel; the HTML line also

  reports "(Mermaid N blocks / M images)".

- **Limits**:
- only files **already saved locally** (an untitled document or a `wsl://` remote file has no

  "same folder" and is refused);

- the system WebView2 Runtime is required; an old Runtime without Print-to-PDF support reports a

  clear error;

- images are given at most 3 seconds before export; the page gets 15 seconds to respond before a

timeout is reported; only one export can run at a time ("An export is already running…").

> 🖼 **Figure 3-3** — The Export PDF / Export HTML context menu and the success line in Output

3.6 Large files and performance

- Documents of 4 MB or more drop syntax highlighting (noted in the status bar, saving memory); preview rendering happens in a separate process with a 250 ms debounce, and the Mermaid engine loads on demand — a large document never blocks your typing.
- Scroll sync is line-anchored, so it stays accurate in very long documents.

3.7 Markdown FAQ

- **Math not rendering?** Make sure you used one of `$…$`, `$$…$$`, `\(…\)`, `\[…\]`. A `$` glued to CJK characters or full-width punctuation may be treated as plain text — `$$…$$` or `\[…\]` is safer.
- **An image is missing?** Relative paths resolve against **the document's own folder**; check that the file is really there.
- **Where is the export button?** In the three context menus — there is no toolbar button.
- **Can I change paper size or margins?** Not yet; it is fixed to A4 (there is no options dialog).

---

4. Image Editor (Image Viewer plugin)

4.1 Opening files and supported formats

- Double-click or right-click open `.png` `.jpg` `.jpeg` `.bmp` `.gif` `.webp` `.tif` `.tiff` and the viewer takes over; if the image cannot be decoded it falls back to the text/binary view instead of pretending.
- You can also use **Tools** → **Open with Image Viewer…**, or right-click a file in the editor and pick **Open with Image Viewer**.
- Note: only the **first frame** is decoded (GIF / WebP animation does not play), and images above 200 megapixels are refused.

4.2 Viewing: zoom / pan / page through a folder

- **Wheel** zooms around the **cursor** (about 1.15× per notch), from **2% to 3200%**.
- **Pan** by dragging with the left button, dragging with the middle button, or holding Space and dragging; the image cannot be dragged out of view.

- **Double-click** toggles between fit-window and 1:1; the toolbar also has

  Fit Window (F) and **1:1 Actual Size (key 1)**.
- ←/→ cycle through images of the same extension set **in the same folder** (wrapping around).
- The status line shows the file name, width×height, zoom percentage, brightness/contrast/saturation

  and a "●Unsaved" marker when there are pending changes.

>  **Figure 4-1** — Image Viewer: toolbar, status line and a zoomed screenshot

4.3 Adjustments

Brightness `-/+` (−100% … +100%), Contrast `-/+` (0.1 … 3.0) and Saturation `-/+` (0 … 3.0) move in 5% steps; **Reset Colors (R)** restores everything (`Ctrl+R` works too).

4.4 Crop and save

- **Crop**: click **Crop: drag to select (C)**, drag the area (the status line says "Drag to select the crop area;

  Enter applies, Esc cancels"), press `Enter` to apply or `Esc` to cancel. A selection smaller than 2×2 pixels does nothing. **Crop cannot be undone** (cropping again crops the result).
- **Save (S)**: overwrites the original file (written to a temp file first, then swapped in; JPEG is

  saved at quality 0.92).
- **Save As…** (toolbar button): PNG / JPEG / BMP filters, with the encoder chosen by the target

  extension (gif / tif / tiff also work).

4.5 Keyboard shortcuts

Key	Action
`F`	Fit window (only scales down)
`1`	1:1 actual size (and centers)
`R` / `Ctrl+R`	Reset colors
`C`	Crop: drag to select
`Enter` / `Esc`	Apply crop / cancel crop
`S`	Save (overwrites the original file)
Toolbar "Save As…"	Save As (the host owns `Ctrl+Shift+S`, so this one has no shortcut)
`←` / `→`	Previous / next image in the folder

Space + drag	Pan

4.6 Limits

- No context menu; no rotation, no multi-image or slideshow mode; crop cannot be undone; GIF/WebP

  show only the first frame.

- When the window gets narrow the toolbar icons shrink step by step, but buttons are **never

  hidden** (each one has a tooltip).

---

5. PDF Editor (PDF Reader plugin)

5.1 Opening a file and the UI

- Double-click any `.pdf` to open it in an editor tab (the file header is checked; a fake PDF that

  is really a text file falls back to the text view).

- **Toolbar** (self-drawn icons, left to right): Undo, Redo, Select, Underline, Strikethrough,

  Highlight group ▾, **Shape group** ▾, **Text annotation group** ▾, Color ▾, Find, Copy, Save ▾ (with Save As), Full screen. When the window narrows, icons shrink (16 → 9 px) rather than disappearing.

- **Bottom status strip**: on the left, the sidebar toggle and the pager (first page / previous /

  page box "n / N" / next / last page); in the middle, status messages such as "Saved", "Selected N characters", "Annotation tool: Rectangle (drag to draw)"; on the right, the quick/deep-read status area, the day/night page color toggle, zoom out, the zoom percentage ▾, zoom in, **Paper quick read** and **Paper deep read**.

- **Left sidebar, three tabs**: **Thumbnails** (click one to jump; the current page has an accent

  frame), **Bookmarks** (a tree; click the text to jump) and **Annotations** (grouped by page; click an entry to select the annotation and jump to it). Drag the sidebar's right edge to resize it (double-click the splitter to reset) or collapse it entirely.

- **Menu commands**: these ten (Save / Undo Annotation / Redo Annotation / Fit Width / Fit Page /

  Zoom In / Zoom Out / Find… / Next Match / Previous Match) live in the **Tools** menu with labels like "PDF: Save", and they act on the focused PDF view.

> **Figure 5-1** — The PDF reader: thumbnails sidebar, toolbar, page and status strip

5.2 Reading: scroll / zoom / find / select text

- **Continuous scrolling**: the plain wheel scrolls, `Shift+wheel` scrolls horizontally, and `Ctrl+wheel` zooms (±15% per notch).
- **Zoom**: 10%–800%; the toolbar's zoom in/out buttons step by ×1.25 each. Clicking the zoom percentage button itself returns to 100%, and its `▾` menu offers **Fit width / Fit page / Actual size**. **A freshly opened document starts at "Fit width".**
- **Paging**: the pager buttons (their tooltips name Home / PageUp / PageDown / End); clicking the page box in the status strip lets you **type a page number** (Enter jumps, Esc cancels, the value is clamped to the valid range).
- **Keyboard**: `←/→` previous/next page, `↑/↓` scroll 60 px, `PageUp/PageDown` scroll 90% of the viewport, `Home/End` jump to the start/end of the document.
- **Find**: the toolbar's Find button (or the menu item **PDF: Find…**). The find bar sits under the toolbar; its buttons (right to left) are `×` close, previous, next, word and case toggles. Enter = next, Shift+Enter = previous, `Esc` closes the bar, `F3` / `Shift+F3` also work. All hits are tinted pale yellow with the current one in orange, and the bar shows "i / N" (past 5,000 hits it truncates and appends a `+`). **Note: `Ctrl+F` belongs to the host** (that is the editor's find) — inside a PDF use the toolbar button or the menu.
- **Text selection**: drag to select (across pages), double-click to select a word (a whole run for Chinese), `Ctrl+A` selects the current page, `Esc` clears the selection. The status bar then says "Selected N characters (Ctrl+C to copy)", and `Ctrl+C` or the toolbar's Copy button puts the text on the clipboard.
- **Page colors**: the day/night button on the status bar inverts the **PDF pages (thumbnails included) **for night reading; it does** not** touch the editor's light/dark theme.
- Encrypted PDFs report "document is encrypted (password entry is not supported yet)".

> 🖼 **Figure 5-2** — The find bar with hits highlighted, and the page-number box

5.3 Annotations: six tool groups

The toolbar tools come in six groups (hover for a full tooltip; once a tool is selected the status bar explains how to draw with it):

Group	Tools and how to use them
Select	Drag to select text / click an annotation; draws nothing
Underline	Select text and the underline is applied automatically (works across pages)

Strikethrough	Same, with a strikethrough
Highlight group ▾	**Text highlight** (select text), **Area highlight** (drag a rectangle), **Freehand highlight** (hold and scribble like a marker). Area and freehand highlights also work on image-only PDFs with no text layer
Shape group ▾	**Line / Rectangle / Ellipse / Cloud** (drag to draw), **Polygon** (click to add points, double-click or press Enter to finish, Esc cancels), **Freehand pen**
Text annotation group ▾	**Text note** (select text first to attach a quoted note; or click a page to drop an empty note and type immediately), **Text box** (drag a box, or click for a default 180×40 pt box), **Callout** (drag from the point being pointed at to where the box should sit; a plain click places a fixed box at the upper right with the leader line drawn for you), **Sticky note** (click a page to drop a "note" icon; double-click to edit)

- Drafts that are too small are treated as mis-clicks and discarded (rectangle/ellipse < 3 pt,

  line < 4 pt, freehand < 2 points, polygon < 3 points); while drawing you see a dashed preview and `Esc` cancels.

- **Color**: clicking the color button cycles through presets; its `▾` menu is the palette —

  Yellow / Green / Blue / Pink / Orange and **Custom…** (a self-drawn picker with a color square, hue bar, old/new preview and a hex readout). If an annotation is selected, the color applies to it; otherwise it applies to the next new annotation.

- **A selected annotation** gets a small toolbar next to it: **recolor / edit / delete**.

  Edit opens a small input box (Enter confirms, Esc cancels, clicking elsewhere also confirms); an empty text box or sticky note created and then confirmed/cancelled without typing is removed automatically, so no empty shells are left behind.

- **Undo / Redo**: 50 steps, covering create / delete / recolor / edit; the toolbar buttons grey out

  accordingly (see 5.7 — the host owns those key combinations, so use the buttons).

- Hovering an annotation shows a bubble with its author and contents.
- **Saving back**: the toolbar's Save (grey when unchanged) or the menu item **PDF: Save** —

  written to a temp file and then swapped in. **Save As…** lives in the Save button's `▾`. A document with unsaved changes shows the dirty marker on its tab.

> **Figure 5-3** — The three annotation group menus expanded, and the mini toolbar on a selected annotation

5.4 Paper quick read / deep read (AI)

Both buttons live at the **far right of the status bar** with tooltips, and the one currently running lights up with the accent color.

- **Before you start**: configure a default model (endpoint + API key) under Settings → Models.

  A missing model, a missing key, or a PDF with no extractable text (a scan) each produce a clear message.

- **Paper quick read**: deliberately restrained (1,500–3,000 characters) and **not an abstract** —

  jargon explained in plain words, intuition for the core formulas, and a "where to look next" section for background. It runs as a single call; very long documents degrade to "per-chunk key points → synthesis" in two passes.

- **Paper deep read**: it **actually fills in the background** — prerequisite knowledge (what it is,

  why it is needed, its role in the paper) and prior work named by reference (who did what, how it differs). Formulas are typeset with `\(…\)` / `\[…\]` and **explained symbol by symbol**; Mermaid diagrams (flowchart / sequence / state / workflow) are added where they help. It runs in stages: reading map → per-chunk deep write → per-chunk review (patch-based) → assembly → gap filling (guaranteeing a structure overview diagram plus the "worth a deep read?" and "reproduction checklist" sections).

- **Product**: `Documents\TruthPad\PDF\<pdf-name>-<identity-hash8>-speedread.md`

  (deep read: `-deepread.md`). Identity prefers the PDF's embedded `/ID` (incremental updates — annotating, bookmarking or marking the file **do not** change it); without an `/ID` it falls back to a fingerprint of page count + page sizes + title/creation date. **The same document is recognised even if you rename the file**, and an existing draft opens with 0 model calls.

- **While it runs**: the status area shows a spinner, the current stage, and "elapsed m:ss"; the

  draft streams into the right-hand **Preview** pane (title marked "generating…"), and the finished document replaces it.

- **Cancel / switch**: clicking **the running one** cancels (cooperatively: it stops at a chunk or

  stage boundary, and the in-flight model call is discarded rather than saved); clicking **the other** stops the current run and automatically starts the new mode once it has settled.

- **Where it opens**: normally in the right-hand **Preview** pane. If that pane is busy with

  another generating task it is not preempted — the document opens in the editor area instead and the status area says so.

- **Logs**: `%LOCALAPPDATA%\TruthPad\pdf.log`, mirrored into the bottom **PDF** output channel

  (each model call logs byte counts, HTTP status and elapsed time).

- **Limits**: text extraction is capped at roughly 400 pages / 400,000 characters (the document

  says so when truncated); scanned PDFs with no text layer fail with an explanation.

> **Figure 5-4** — The spinner and stage text in the status bar, and a deep-read draft in the right

> preview

5.5 Where the AI configuration comes from (Settings → Models)

- The store is `%APPDATA%\TruthPad\models.json`, **shared by every plugin that needs AI** (PDF quick/deep read, browser translation and subtitle-to-article).
- Settings → Models lets you keep several models; each card has **ID / Name / Vendor / Endpoint / API key**, plus a** Default **radio, a** Test **button and** Delete**. Context 1M and max output 384K are fixed read-only specs, and the** key is stored in plain text in models.json**.
- **Test** sends one minimal request with that model and fills the result in place (✓ OK / ✗ Failed).
- **Save model config** broadcasts to the loaded plugins immediately — **no restart**; invalid JSON is refused and the file is left untouched.
- **Open in notepad** edits the file directly; the settings page re-reads it when you open it.

5.6 Performance settings (the plugin's ten options)

Configure them from the plugin manager's **Configure** dialog for "PDF Reader" (all take effect immediately except the MuPDF cache budget):

Option	Default	Range	Meaning
MuPDF cache budget	32 MB	4–1024	Per-document MuPDF store budget; applies to the next document you open
Shrink cache on page turn	50%	0–90	Shrink the cache to this percentage when you turn a page (0 = never)
Page bitmap memory budget	64 MB	8–2048	Cap for rendered page bitmaps; the least recently used are dropped, the current page never is
Page bitmap cache entries	8	1–128	Roughly the current page plus one page of prefetch in each direction
Display list cache entries	3	1–64	How many parsed display lists to keep
Full-page render limit	12 MP	1–256	Above this, only the visible band is rendered (memory drops from page area to viewport area, sharpness unchanged)
Pages sized at open	32	1–4096	Page sizes resolved up front; the rest are estimated and resolved as you scroll near them
Size resolve batch	32	1–4096	How many unresolved page sizes to resolve at once

Band render margin	1	0–8	Extra viewport heights rendered above/below the visible area in band mode
Thumbnail cache entries	40	0–1000	How many sidebar thumbnails to cache; 0 = no cache

5.7 Shortcuts and limits

Key	Action
`Ctrl+wheel` / toolbar buttons	Zoom (10%–800%)
`←` `→` / `PageUp` `PageDown` / `Home` `End`	Previous/next page / page up-down / start-end of document
`↑` `↓`	Small scroll steps
`Ctrl+A` / `Esc`	Select current page / clear selection
`Ctrl+C`	Copy the selected text
`F3` / `Shift+F3`	Find next / previous

- Undo, redo and save are done with the **toolbar buttons** or the menu — the host owns `Ctrl+Z` / `Ctrl+Y` / `Ctrl+S`, so those combinations do nothing inside a PDF view (`Ctrl+C`, `Ctrl+A` and `F3` are unaffected);
- **There is no context menu** (everything lives on the toolbar and the mini toolbar on a selected annotation);
- `Ctrl+F` belongs to the host, so use the toolbar's Find or the **PDF: Find…** menu item;
- encrypted PDFs cannot take a password yet.

---

6. Built-in Browser

6.1 Opening it and the UI

- **Entry points**: the **Browse** button at the right of the title bar, or **Tools → Open Embedded Browser**. One browser view is one editor tab (the view has no tab strip of its own); the page title becomes the tab title.
- **Top bar** (five icon buttons plus the address bar): Back (enabled only when possible), Forward, Reload, **Subtitles**, **Run JavaScript**, then the address bar filling the rest.
- **Address bar**: focusing it drops an autocomplete list (up to 8 entries; history first, then

frequent site domains). Enter navigates — a URL with a scheme opens as is, a Windows path (`C:\…` or a UNC path) becomes `file://`, and anything else gets `https://` prepended.

- **Start page** (new tab): a search box (non-URL input goes to Google search), **Recently visited** and **Favorites**, following the UI theme and language.
- **Link behaviour**: `target="_blank"`, `window.open` and middle-clicks open a **new browser tab** instead of replacing the current page.

> **Figure 6-1** — A browser tab: the five top-bar buttons, the address bar and the start page

6.2 Browsing and enhancements

- **Ad blocking** (on by default, request level): a built-in list of ad/tracking hosts; a match gets an empty response and never leaves your machine. **Extra hosts to block** appends your own (comma or newline separated; subdomains count). **Log blocked requests** writes "ad filter: N blocked" to the log.
- **Cosmetic filtering** (on by default): injects conservative CSS to hide ad slots (never danmaku or the video itself) and auto-clicks YouTube's "Skip ad".
- **Page script injection** (master switch, on by default): turning it off injects nothing at all — **subtitles and the userscript stop together**.
- **Userscript** (off by default): point it at a `.js` file and it is injected into every page you open (a mini Tampermonkey). **Userscript hosts** is a comma-separated list of substrings (e.g. `bilibili.com,youtube.com`); empty means every site.
- **Run JavaScript** dialog (the fifth top-bar button): type some JS, then **Run once** for the current page or **Save as auto-inject** to write it into the userscript file.
- If WebView2 cannot start, the content area shows the error (usually it means the Microsoft Edge WebView2 Runtime needs to be installed).

6.3 Subtitles: Chinese captions for videos

Site support: bilibili is the fully verified pipeline (multi-part videos, BV IDs and resume all work); any page with a `<video>` element goes through the generic fallback; YouTube capture is **experimental and currently shelved**.

One-time preparation:

- **ASR model**: click **Download ASR model** in the plugin settings (or the subtitle settings

dialog). The default is `large-v3-turbo` (about 1.6 GB, the best quality/speed balance); other choices are tiny.en / base.en / small.en / medium.en. The download source can be `huggingface.co` or `hf-mirror.com`. Models live in `%LOCALAPPDATA%\TruthPad\asr`.

- **DeepSeek key**: **Shift+click the Subtitles button** in the top bar to open the subtitle settings, paste your `sk-…` key and press Save (the key is used only to send subtitle **text** for translation — **audio never leaves your machine** — and it is stored locally encrypted with Windows DPAPI).

Generating subtitles: open the video page and press **Subtitles**. Depending on the current state one of these happens:

- no model yet → the subtitle settings dialog opens and asks you to download one (nothing starts);
- no key → subtitles are generated without translation, with a persistent hint in the overlay;
- this episode is already generating → you are told to wait (to start over, use the context menu item **Regenerate Subtitles**);
- this episode is **already fully transcribed** → it is shown as is, **nothing is regenerated**;
- it is only partially covered (say, you left halfway once) → **it continues from the coverage point**; already-transcribed parts are never redone;
- the audio temp files are still there and cover enough → the audio is reused and only the missing segments are transcribed.

The overlay while it runs (it floats above the page and does not shrink it):

- one original line plus one Chinese line (for Chinese audio there is a single line and only proofreading is done);
- during generation the Chinese slot shows progress: "⏳ stage… detail percentage (elapsed N s)";
- the bottom-right corner shows "n / N", with a "✓" once whole-episode review is done; incomplete coverage leaves a persistent "⚠" warning line;
- **drag the bar up and down** to change where it docks (the position is remembered);
- **context menu**: Copy Current Line / Generate Article (or **View article** + **Regenerate Article **once one exists) / Regenerate Subtitles (or** Generate subtitles** when there are none) / Subtitle Settings / Hide subtitles.

Subtitle settings (Shift+click Subtitles, or the context menu item) also let you change:

- **Files & cache** (all local): model info with **Open model folder / Delete model / Download model**; cache info with** Open cache folder / Clear subtitle cache**; temp files with** Open temp folder / Clean temp files**;
- **Subtitle appearance**: original color / Chinese color / background color (`#RRGGBB` boxes plus

Pick… swatches), a background opacity slider, and **five one-click presets** (YouTube classic / Cinema (yellow on black) / White outline, no background / Light theme / High contrast (black on yellow)) that apply immediately;

- the **Live article rendering** checkbox (off by default; see 6.4).

Taking a line into your notes: the global hotkey `Ctrl+Alt+S` inserts the current cue at the editor caret, formatted according to **Insert-into-note format** (bilingual / translation only / original only).

Export: the plugin's action buttons **Copy this episode as SRT** (a standard SRT timeline with bilingual lines) and **Copy this episode as text** (one original line, one Chinese line; Chinese audio yields Chinese only) — both go through the clipboard, nothing is written to disk.

Cache and temp files:

- subtitle cache `Documents\TruthPad\Browser\subs\<site>_<video>.json` (original + translation,

  written as each segment finishes, so nothing already transcribed is lost if you quit early);

- temp audio `%TEMP%\tp-browser` (named per video, kept for resume/reuse; use **Temp file

  retention (days)** or clean it by hand any time).

Interruptions: closing the tab, following a link away, or switching videos stops the running task (the in-flight model call finishes first); coming back to the episode continues from where it stopped.

> **Figure 6-2** — The subtitle overlay (original + Chinese, n/N at bottom right) and the appearance section of the subtitle settings dialog

6.4 Subtitles → article (lecture notes)

- **Entry**: the overlay's context menu → **Generate Article**; once an article exists the menu shows

  View article (opens it instantly, 0 model calls) and **Regenerate Article**.

- **Writing style** (plugin setting **Article style**): **Auto detect** (default — the content type

  is sampled first, and if that fails it falls back to Academic lecture) / **Academic lecture** / **Light essay** / **News report**.

- **The pipeline** (as you see it — the overlay's progress line advances stage by stage):

1. **Detect the content type** (auto style only; one sampled call);
2. **Write the skeleton**: document title, a global symbol table, and per-section headings with

   time ranges, key points and a formula outline;

3. **Write each section in full**, slicing the transcript by the skeleton's time ranges and

   stitching to the previous section's ending;

4. **Review each section**: check against the skeleton and transcript, de-spoken-ify the prose, normalise formulas, repair diagrams that cannot render;
5. **Write the overview and the summary**, assemble overview → sections → summary, then run a whole-document hard check.

- **Product**: `Documents\TruthPad\Browser\<video title>-<style>.md` (a fixed name — **regenerating overwrites it**; UTF-8 with BOM, CRLF). Per-section checkpoints live in `Browser\articles\<key>.json`.
- **Resume**: after a cancel or a crash, pressing **Generate Article** again **continues from the first unfinished section**; finished sections are not re-requested. A completed article opens with 0 model calls.
- **Live rendering** (off by default): with **Live article rendering** enabled the article streams into the right-hand **Preview** pane from the skeleton stage on (long articles can feel janky, hence the default). With it off, no partial file is written and the right pane stays put.
- **Where it opens when done**: preferably in the right-hand **Preview** pane. If that pane is busy with another **generating** task it is not preempted ("Right pane is busy with another task…"); the article still finishes, and you can open it later via **View article**.
- **Writing rules the plugin enforces**:
- formulas **only** as `\(…\)` (inline) and `\[…\]` (display) — any `$…$` is repaired automatically, and no stray `$` may appear outside code fences;
- **every heading carries a timestamp** so the note maps back to the video;
- length is **at least 2.6× the transcript and uncapped** (better long than short, but padding and repetition are forbidden);
- illustrations prefer inline Mermaid (plain text, rendered natively by the preview); a real image is inlined only when the model is confident about a public direct link — no local images, no base64, no inline SVG.

> 🖼 **Figure 6-3** — An article refreshing live in the right preview, and the heading hierarchy with timestamps in the finished note

6.5 Browser plugin settings

Click **Configure** for "Browser" in the plugin manager:

- **Info & actions (11)**: model files (info), **Download ASR model**, Open model folder, Delete

model files, subtitle & translation cache (info), Open cache folder, Clear subtitle cache, Open temp audio folder, Clean temp files, **Copy this episode as SRT**, **Copy this episode as text**.

- **Injection & userscript (4)**: Page script injection (on), Enable userscript (off),

  Userscript path, Userscript hosts.

- **Subtitles (about 16)**: Subtitles (on), Per-video cap (0 = whole video), Whisper model

  (large-v3-turbo), Threads (8), Chunk seconds (45), Whole-episode review (on), Subtitle display (bilingual / translation only / original only), Subtitle font size (16), Insert-into-note format (bilingual), Background opacity (75), Original text color (`#FFFFFF`), Chinese text color (`#6CB6FF`), Subtitle background color (`#000000`), Subtitle dock position (100 = bottom), Subtitle alignment (center), Model download source (huggingface.co), Temp file retention (days).

- **Article (3)**: Article style (Auto detect), Cues per article segment (180, range 40–400),

  Live article rendering (off).

- **Ads (4)**: Ad blocking (on), Cosmetic filtering (on), Extra hosts to block (empty),

  Log blocked requests (on).

Almost everything applies immediately (the subtitle appearance is read fresh on every paint).

6.6 Known limitations

- **YouTube capture is experimental and currently shelved**: YouTube's direct audio URLs return 403

  on replay, so only the in-page MSE capture route works. For sites other than bilibili, expect the generic fallback (any page with a `<video>` element).

- The web area has no custom context menu (you get WebView2's own).
- Chinese audio produces a single subtitle row and is only proofread — that is by design, not a

  missing feature.

- Subtitle quality depends on the audio and the model: `tiny.en` is fast but error-prone, while

  `large-v3-turbo` is the default balance.

- Where a plugin manifest's wording disagrees with the actual paths, **the actual path wins**: the

  subtitle cache really lives in `Documents\TruthPad\Browser\subs` (not `%LOCALAPPDATA%`).

Appendix A Shortcut summary

Host application: see 2.13.

PDF Reader: see 5.7.

Image Viewer: see 4.5.

Browser (global hotkey, available at any time):

Key	Action
`Ctrl+Alt+S`	Insert the current subtitle cue into the editor (per Insert-into-note format)

Appendix B Data and file locations

Content	Location
Main configuration (layout, sidebar width, output level, language…)	`%APPDATA%\TruthPad\config.ini`
Colors	`%APPDATA%\TruthPad\theme.json`
Model configuration (shared by AI plugins)	`%APPDATA%\TruthPad\models.json`
Application and plugin logs	`%LOCALAPPDATA%\TruthPad\` (`plugins.log`, `preview.log`, `pdf.log`, `browser.log`, `search.log`, `aicfg.log`, `theme.log`, `config.log`, `crash.log`)
Web runtime data	`%LOCALAPPDATA%\TruthPad\WebView2` (host), `…\WebView2-browser` (browser plugin)
Whisper models	`%LOCALAPPDATA%\TruthPad\asr\ggml-*.bin`
Subtitle cache (original + translation)	`Documents\TruthPad\Browser\subs\<site>_<video>.json`
Article checkpoints	`Documents\TruthPad\Browser\articles\<key>.json`
Generated lecture notes	`Documents\TruthPad\Browser\<video title>-<style>.md`
PDF quick / deep reads	`Documents\TruthPad\PDF\<pdf-name>-<hash8>-speedread.md` / `-deepread.md`
Temp audio (safe to clean)	`%TEMP%\tp-browser`

Appendix C FAQ

- **Right-clicking a file does not open minimal mode?** The association command was changed. Run

  Help → Repair Open-with Associations (current user only).

- **The Markdown preview is blank or will not open?** The preview depends on the Microsoft Edge WebView2 Runtime; install it and retry (the preview pane tells you the same thing).
- `Ctrl+F` **does nothing inside a PDF?** That shortcut is the editor's find. Use the PDF toolbar's Find button or the **PDF: Find…** menu item.
- **The model card's Test fails?** Check the endpoint (it must be an OpenAI-compatible `/chat/completions` URL) and the key. Invalid JSON is refused on save and the file is left alone.
- **I clicked quick/deep read and nothing started?** One of three things: no default model configured, the model has no API key, or the PDF is a scan with no text layer. All three show an explicit message.
- **Subtitles are original-only, no Chinese?** Either the DeepSeek key is not set yet, or the audio is Chinese (which is a single-row, proofread-only case by design). Set the key and press **Subtitles** again to continue with translation.
- **Subtitle generation is slow?** Transcription runs locally: bigger models are more accurate but slower. Try a smaller model or adjust **Threads**. Long videos are transcribed in segments as they arrive, so the first minutes appear quickly.
- **The generated article / read did not appear on the right?** That pane may be busy with another **generating** task (they never preempt each other). Wait for it to finish, or open the product from its own entry point (the overlay's **View article**, or clicking quick/deep read again).
- **The exported HTML has no contents sidebar?** By design: the exported HTML is a **single static page** with no scripts; use the in-editor preview for the interactive contents sidebar.

Appendix D Screenshots to be added

Every placeholder in the text can be captured in one pass from this list. Suggested target folder: `web/assets/manual/`, dark theme throughout (matching the website shots), and the same resolution as the website assets.

Figure	Content	Suggested capture
1-1	Minimal mode: a single open .md tab	Associate a test file and launch with `--quick`
1-2	Markdown with math and Mermaid in split preview + the export menu	Open a sample with split preview, capture, then overlay the menu

1-3	Plugin manager (lazy / state badges)	Open the plugin manager page
1-4	PDF reader + a speed-read draft in the right Preview pane	Open a real paper and run a quick read
1-5	A bilibili lecture playing with Chinese subtitles + the note on the right	Run subtitles on a public teaching video
2-1	Welcome page (Get Started / Recent / Shortcuts)	Launch normally (no file)
2-2	UI overview (title bar, icon rail, tab bar, status bar annotated)	Open a workspace and a markdown file
2-3	The drop hint while dragging a tab to split	Screen-capture mid-drag
2-4	Search results in the sidebar (grouped + replace row)	Search a keyword in the sample repo
2-5	Workspace tree context menu + the drag card	Right-click a tree node (the card needs a timed capture)
2-6	Terminal panel (cmd / PowerShell / SSH / TTY) and the SFTP file sidebar	Open the Terminal panel
2-7	Settings: Appearance and Models sections	Open the settings page
2-8	Plugin manager + a plugin settings dialog	Click Configure on a plugin
3-1	Split mode and the two sync arrows on the preview top bar	Open a markdown file's preview
3-2	Math / Mermaid / code highlighting / contents sidebar	Use a sample document with all of them
3-3	The export context menu and the Output success line	Capture before and after clicking Export PDF
4-1	Image Viewer (toolbar + status line)	Open a large image
5-1	The PDF reader as a whole	Open a PDF with bookmarks
5-2	Find bar with highlighted hits; the page-number box	Search a common word in a PDF
5-3	The three annotation group menus expanded + the mini toolbar on a selected annotation	Expand each menu and capture
5-4	The quick/deep-read status area and a generating draft on the right	Run a quick or deep read
6-1	A browser tab (top bar + start page)	Open the embedded browser
6-2	The subtitle overlay and the subtitle settings dialog	Open a video with subtitles, then Shift+click Subtitles
6-3	An article refreshing live in the right preview	Enable live rendering and capture